Programmazione ad aspetti

Considerazioni su Proxy

Affinché un Proxy funzioni e la sua logica venga eseguita al momento giusto, le classi client devono interagire esclusivamente con il Proxy e non direttamente con il RealSubject, altrimenti il meccanismo verrebbe bypassato. Questo solleva il problema pratico di come forzare il passaggio delle chiamate attraverso il Proxy (come nel caso del Reference Monitor). Inoltre, se la funzionalità offerta da un Proxy si rendesse necessaria per molte classi diverse, si dovrebbe implementare un Proxy specifico per ciascuna classe, portando ad una proliferazione ingestibile di classi.

Considerazioni su Object-Oriented

Utilizzando le classiche tecniche Object-Oriented (OOP), l'implementazione di alcune funzionalità sistematiche (chiamate *concern*, come ad esempio il logging) finisce per essere frammentata e sparsa tra molte classi diverse. Questa impossibilità di racchiudere una funzionalità trasversale in un'unica classe, dovuta ai limiti della scomposizione in oggetti, viene definita "*tirannìa della decomposizione predominante*"

Concern

Un concern è fondamentalmente un requisito o un insieme di funzionalità del sistema software (ad esempio: logica di business, sicurezza, persistenza dei dati o logging). Durante la progettazione, è importante puntare alla "separazione dei concern" (*separation of concern*), ovvero isolare le differenti realizzazioni dei requisiti in moduli separati, in modo che ogni funzionalità abbia il proprio modulo dedicato.

Case study su concern

Uno studio effettuato sul codice di Apache Tomcat ha evidenziato come alcuni concern (come il parsing XML) siano ben organizzati in singole classi. Tuttavia, il codice relativo ad altri concern, come il logging o la scadenza delle sessioni, risulta sparso (*scattered*) in moltissime classi differenti. Questi tipi di funzionalità che tagliano trasversalmente il sistema sono chiamati concern trasversali (*crosscutting concern*). L'effetto visibile nel codice è il *tangling* (aggrovigliamento), ovvero la situazione in cui diversi concern sono mischiati all'interno dello stesso modulo.

Esempio

Il seguente esempio illustra un sistema software con due requisiti:

1. mantenere i dati relativi a vari prodotti.
2. registrare ogni volta che il costo di un prodotto subisce una variazione.

Per realizzare questo con la OOP classica, vengono individuate due classi distinte: Logger e Product.

```
// Logger: classe che tiene i log
public class Logger {
        public void writeLog(String l) {
            // ...
```

```
        }
}
```

Classe Product

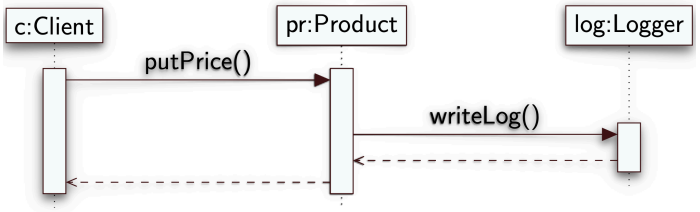
Nell'implementazione OOP proposta, la classe Product deve contenere al suo interno il riferimento e il codice per il logging. In particolare, il metodo putPrice() di Product non si limita ad aggiornare il prezzo, ma deve istanziare e richiamare esplicitamente il metodo log.writeLog() della classe Logger.

```
// Product: classe che tiene informazioni sui prodotti
public class Product {
        private double price;
        private Logger log;        // codice per Log

        public Product() {
                log = new Logger();      // codice per Log
                price = 0.0;
        }

        public void putPrice(double p) {
                log.writeLog("Price changed from "+price+" to "+p); // cod
ice per Log
                price = p;
        }

        public double getPrice() {
                return price;
        }
}
```



Considerazioni

Il codice della classe Product evidenzia i limiti della OOP. La classe infatti dovrebbe occuparsi solo dei dati del prodotto, ma è costretta ad avvisare il Logger dei cambiamenti. Questo porta allo scattering (il requisito di registrazione è diviso tra le due classi) e al tangling (il codice di logging si mischia alla logica del prodotto). Per una progettazione pulita, la classe Product non dovrebbe contenere alcuna invocazione ai metodi di Logger, cosa che però le normali tecniche Object-Oriented non riescono ad evitare in modo naturale.

Soluzione Aspect-Oriented

Nella soluzione Aspect-Oriented, la classe che gestisce la logica di business (ad esempio Product) deve essere completamente priva di interazioni con le classi che gestiscono funzionalità di contorno (come Logger). L'Aspect-Oriented Programming (AOP) risolve questo problema permettendo di separare i concern trasversali definendo un **aspetto**. Un aspetto è un modulo che specifica quali azioni compiere al verificarsi di determinati eventi nel codice. È strutturato componendo *advice* e *pointcut*, i quali si collegano a punti specifici del programma chiamati *join point*.

Join point

Un join point è un punto ben preciso all'interno dell'esecuzione di un programma in cui un aspetto interviene per iniettare il suo comportamento (il crosscutting concern). Tipi comuni di join point includono: l'invocazione o l'esecuzione di un metodo, la costruzione di un oggetto tramite `new`, l'acceso in lettura o scrittura a un campo di una classe, il lancio di un'eccezione o l'esecuzione di test condizionali.

Pointcut

Il pointcut è il costrutto logico che si occupa di selezionare i join point e di raccogliere il contesto di esecuzione. Per chiarire la differenza: il pointcut è la regola formale di selezione (ad esempio, "intercetta tutti i costruttori che prendono un intero come argomento"), mentre il join point è l'effettivo evento a runtime che soddisfa tale regola.

Advice

L'advice è il codice che viene materialmente eseguito quando il programma raggiunge un join point selezionato da un pointcut. Il suo contenuto assomiglia al corpo di un metodo tradizionale, poiché incapsula la logica desiderata. A seconda di come viene definito, il codice di un advice può essere eseguito prima, dopo, o persino al posto (sostituendolo) del join point catturato.

Aspetto LoggerAsp

Il frammento di codice LoggerAsp è l'aspetto pratico che implementa l'avvio del logging senza sporcare la classe Product. Al suo interno si occupa di:

→ creare l'istanza della classe Logger.

→ definire un **pointcut** (loggable()) che intercetta esplicitamente la chiamata al metodo Product.putPrice(*).

→ definire un **advice** (nell'esempio un blocco before()) contenente il codice da eseguire (cioè l.writeLog) ogni volta che si verifica la cattura definita dal pointcut (ogni volta che viene chiamato putPrice()).

```
// Aspetto che implementa le operazioni per avviare il logging
public aspect LoggerAsp {   // dichiarazione aspetto
        // crea una istanza che contiene i dati per il logging
        private Logger l = new Logger();

        // definisce il pointcut, ovvero i punti da catturare (intercettar
e)
        pointcut loggable() : call(public void Product.putPrice(*));

        // definisce l'advice, ovvero le operazioni da eseguire quando avv
iene
```

```
        // la cattura definita dal pointcut
        before() : loggable() {
                l.writeLog("Price changed");
        }
}
```

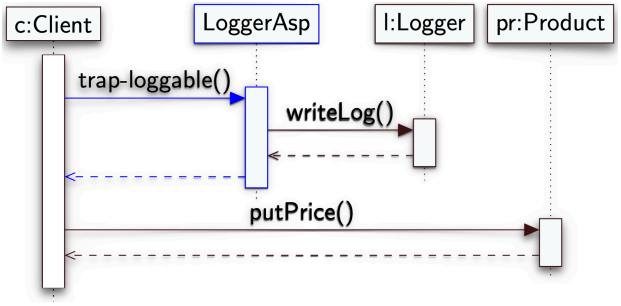
La classe Product si riduce semplicemente come segue:

```
public class Product {
        private double price;

        public Product() {
                price = 0.0;
        }

        public void putPrice(double p) {
                price = p;
        }

        public double getPrice() {
                return price;
        }
}
```



Altri concern

Se ad un sistema si volessero aggiungere altri requisiti (come misurare i tempi di variazione dei costi, implementare l'autenticazione tramite access control o tenere traccia del numero di prodotti), l'utilizzo della sola programmazione ad oggetti (OOP) porterebbe a gravi problemi. Si verificherebbe un'esplosione del numero di classi o, alternativamente, il codice dei vari concern si mischierebbe in modo disordinato all'interno della stessa classe. Di conseguenza, il codice originario diventerebbe complesso, difficile da comprendere e da modificare, più soggetto a errori e non riutilizzabile. L'AOP nasce proprio per evitare questi difetti legati all'implementazione Object Oriented dei concern crosscutting.

Esempio

Per riassumere le interazioni tra join point, pointcut e advice, mostriamo l'esempio di una classe Circle dotata del metodo getRadius():

→ **join point**: l'esecuzione vera e propria del metodo getRadius() all'interno del programma.

→ **pointcut**: all'interno dell'aspetto CircleAsp viene definito un pointcut chiamato rad() che applica la regola di cattura execution(public int Circle.getRadius()).

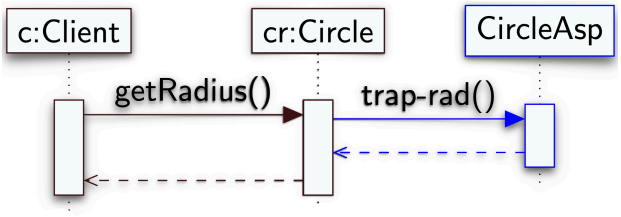
→ **advice**: viene definito un blocco before() : rad() contenente un'istruzione di stampa. Grazie a questa configurazione, l'istruzione di stampa eseguirà sempre un istante prima che parta il codice reale del metodo getRadius().

```
// Aspetto CircleAsp
public aspect CircleAsp {
        // questo pointcut intercetta l'esecuzione del metodo getRadius()
della classe Circle
        pointcut rad() : execution(public int Circle.getRadius());

        // definisce l'advice, ovvero le operazioni da eseguire prima del
join point
        // ovvero prima dell'esecuzione del metodo getRadius()
        before() : rad() {
                System.out.println("prima del metodo getRadius()");
        }
}
```

```
public class Circle {
        private int radius = 5;

        public int getRadius() {
                return radius;
        }
}
```



Tipi di pointcut

I principali tipi di pointcut permettono di selezionare punti differenti nell'interazione con un metodo. Il pointcut call (ad esempio `call(double Product.getPrice())`) seleziona il punto del programma in cui avviene la chiamata al metodo dal lato del chiamante. Al contrario, il pointcut `execution(double Product.getPrice())` seleziona il punto in cui inizia l'esecuzione del metodo dal lato del chiamato. Sebbene spesso siano equivalenti, se è necessario intercettare operazioni su classi di libreria che non possono essere modificate, è possibile usare esclusivamente `call`.

Weaver

Un weaver (come il compilatore AspectJ) è uno strumento utilizzato per fondere insieme gli aspetti e le classi, generando un'applicazione finale che contiene solo costrutti tradizionali. Il weaver

elimina i costrutti del linguaggio per aspetti, producendo un codice sorgente intermedio che viene elaborato da un normale compilatore. Il risultato finale è un bytecode compatibile ed eseguibile da una normalissima JVM standard.

Esempio

Il seguente esempio illustra come implementare un Proxy tramite gli aspetti per controllare gli accessi ai metodi di una classe (Book), mantenendo separati i ruoli. In questo scenario, una classe AuthrzRules contiene le regole di accesso. Viene definito un aspetto chiamato Protection che svolge il ruolo di Proxy (e che potrebbe farlo per molteplici classi contemporaneamente). Questo aspetto intercetta le operazioni di lettura e scrittura definendo dei pointcut e utilizza un advice di tipo around(). All'arrivo della chiamata, l'aspetto prende il controllo, verifica i permessi tramite AuthrzRules e, solo in caso positivo, fa proseguire l'esecuzione del metodo reale invocando proceed(b).

```
public class Client {
        private static AuthrzRules r = new AuthrzRules();

        public static void main(String[] args) {
                Book b = new Book(); // notare che si usa l'istanza di Boo
k
                r.fillRules();
                System.out.println("Client: " + b.getText());
                b.append("Hello world.");
        }

        public static AuthrzRules getRules() {
                return r;
        }
}
```

```
public class AuthrzRules {
        private List<String> rperm = new ArrayList<String>();
        private List<String> wperm = new ArrayList<String>();

        public boolean canRead(Object o) {
                if (rperm.contains(o.getClass().getName())) return true;
                return false;
        }

        public boolean canWrite(Object o) {
                if (wperm.contains(o.getClass().getName())) return true;
                return false;
        }

        public void fillRules() {
                rperm.add("Book");
```

```
        }
}
```

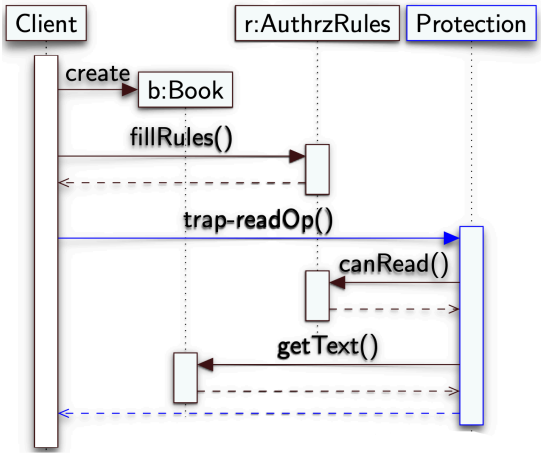
```
public aspect Protection { // Sta facendo da Proxy per la classe Book
        private AuthrzRules r = Client.getRules();

        pointcut readOp(Book b) : call(String Book.getText()) && target
(b); // Il target è la "collezione del contesto"

        pointcut writeOp(Book b) : call(void Book.append(String)) && targe
t(b);

        String around(Book b) : readOp(b) {
                System.out.println("Aspect: before read");
                if (r.canRead(b)) return proceed(b);
                System.out.println("Aspect: Read access has not been grant
ed");
                return null;
        }

        void around(Book b) : writeOp(b) {
                System.out.println("Aspect: before write");
                if (r.canWrite(b)) proceed(b);
                else System.out.println("Aspect: Write access has not been
granted");
        }
}
```



Collezione del contesto

Per applicare logiche complesse (come il controllo degli accessi visto nell'esempio precedente), un aspetto ha bisogno di collezionare il contesto di esecuzione, ovvero di ottenere riferimenti concreti come l'oggetto su cui è stato invocato il metodo. Questo si ottiene aggiungendo il costrutto target() alla regola del pointcut. Ad esempio, l'espressione `call(String Book.getText()) && target(b)` seleziona i

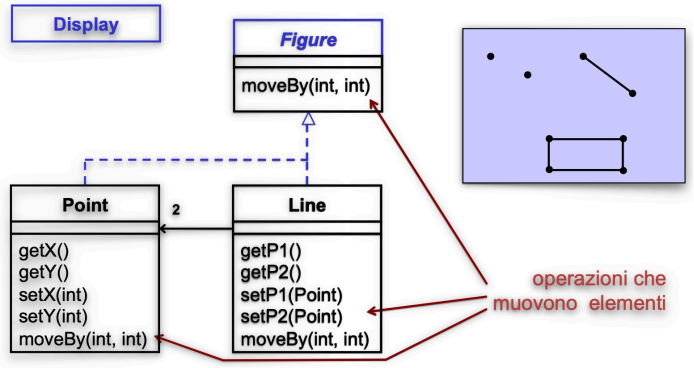
join point in cui l'oggetto chiamato è un'istanza di Book (o una sua sottoclasse) e cattura il riferimento di quell'istanza inserendolo nella variabile b. Tale riferimento catturato viene quindi passato all'advice corrispondente per poterne leggere o manipolare i dati.

Aspect-Oriented-Programming

L'Aspect-Oriented-Programming (AOP) si basa sull'utilizzo degli "aspetti". Un aspetto è un modulo che permette di esprimere uno o più concern trasversali (crosscutting) in modo modulare. L'AOP è particolarmente utile per implementare funzionalità di sistema che tipicamente si sparpagliano nel codice, fornendo esempi classici come la gestione del logging, la sincronizzazione, la sicurezza e la gestione della memoria.

Esempio: editor per figure

Questo esempio illustra un editor geometrico composto da un'interfaccia Figure (dotata del metodo moveBy(int, int)) e da due classi concrete, Point e Line, che la implementano. Entrambre le classi definiscono metodi per maniporare i dati spaziali, come getX(), getY, setX(), setY per il punto, e getP1(), getP2(), setP1(), setP2() per la linea, oltre ai metodi per lo spostamento moveBy(). Il problema architetturale sorge dal fatto che ciascun elemento che viene spostato deve essere necessariamente ridisegnato. Di conseguenza, i metodi di movimento di Point e Line devono invocare un metodo di aggiornamento della classe Display, finendo per mischiare due concern (preoccupazioni) distinte all'interno dello stesso codice.




```
public interface Figure {
        public void moveBy(int dx, int dy);
}

public class Point implements Figure {
        private int x = 0, y = 0;
        public int getX() { return x; }
        public int getY() { return y; }
        public void setX(int x) { this.x = x; }
        public void setY(int y) { this.y = y; }
        public void moveBy(int dx, int dy) { /* codice */ }
}

public class Line implements Figure {
        private Point p1, p2;
```

```
        public Point getP1() { return p1; }
        public Point getP2() { return p2; }
        public void setP1(Point p1) { this.p1 = p1; }
        public void setP2(Point p2) { this.p2 = p2; }
        public void moveBy(int dx, int dy) { /* codice */ }
}
```

Crosscutting concern per Display

Se si utilizza la normale programmazione ad oggetti, l'aggiornamento grafico diventa un crosscutting concern: il codice di metodi come setP1(), setP2() e moveBy() deve contenere esplicitamente l'invocazione di Display.update(). Poiché l'aggiornamento del display non risiede in un suo modulo indipendente, se emergesse la necessità di cambiare le logiche di visualizzazione, bisognerebbe rintracciare e modificare tutte le classi chiamanti. Da un punto di vista concettuale del design, le classi matematiche Line e Point non dovrebbero saper nulla dell'esistenza della classe Display.

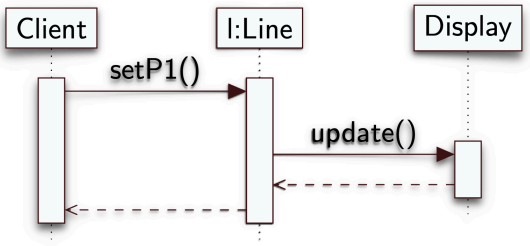
```
public class Line implements Figure {
        private Point p1, p2;

        public Point getP1() { return p1; }
        public Point getP2() { return p2; }

        public void setP1(Point p1) {
                this.p1 = p1;
                Display.update(); <--
        }

        public void setP2(Point p2) {
                this.p2 = p2;
                Display.update(); <--
        }

        public void moveBy(int dx, int dy) {
                /* codice */
        }
}
```



Aspetti

Strutturalmente, gli aspetti possiedono caratteristiche e limitazioni precise all'interno del linguaggio:

- Possono contenere l'implementazione di **campi e metodi** (esattamente come le classi), oltre a costruttori, ma rigorosamente senza argomenti.
- Possono dichiarare la loro **visibilità** tramite modificatori (come public).
- Possono essere definiti **abstract**, in modo da essere riusati da altri aspetti fornendo pointcut o metodi astratti; un aspetto astratto di per sé non intercetta nulla.
- Possono **estendere classi** o **altri aspetti abstract** e possono **implementare interfacce**. Tuttavia, non possono ereditare da aspetti che non siano abstract.
- A differenza delle normali classi, non possono essere istanziati direttamente tramite l'uso della keyword `new`. Il sistema provvede a crearne automaticamente un'unica istanza.

Sintassi dei pointcut

I pointcut sono i costrutti che identificano i join point (i punti di intercettazione) nel flusso del programma e possono essere dichiarati esplicitamente per assegnare loro un nome. La sintassi generale per definirli è:

`pointcut nome([param]) : designator(joinpoint);`

I componenti di questa espressione sono:

→ **nome**: definisce una stringa che fa da identificatore per il pointcut.

→ **designator**: indica la categoria del join point che si intende selezionare durante l'esecuzione. I designatori principali includono **call** (chiamata a metodo), **execution** (esecuzione del metodo), **get** (lettura di variabile), **set** (scrittura di variabile).

→ **joinpoint**: definisce l'evento esatto da intercettare, specificato tipicamente attraverso la signature (firma) di un metodo.

Esempi di pointcut

Per selezionare e intercettare eventi specifici si usano diverse sintassi. Ad esempio, per catturare ogni singola chiamata al metodo setX si definisce:

`pointcut catturaSetX() : call(void Point.setX(int));`

È possibile anche catturare le chiamate basandosi sulle interfacce:

`call(void Figure.moveBy(int, int))`

Il sistema quindi intercetterà l'invocazione di quel metodo su qualsiasi classe che implementi l'interfaccia Figure.

Infine, per intercettare la creazione di un nuovo oggetto (cioè l'istanziazione tramite costruttore), si usa il costrutto new:

`call(Point.new())`

Flusso di controllo

Un pointcut è progettato per catturare il controllo del flusso del programma e far sì che venga eseguito un frammento di codice specifico, chiamato advice. Per esempio, si può definire un pointcut myChange() che intercetta la chiamata a Line.setP1(Point):

```
// pointcut che cattura il metodo setP1 di Line
```

```
pointcut myChange() : call(void Line.setP1(Point));
```

A questo punto, si definisce un advice collegato:

```
// advice che esegue quando il pointcut cattura il flusso
before() : myChange() {
        // corpo dell'advice
        System.out.println("prima");
}
```

Grazie a questa associazione, quando il programma tenta di eseguire il metodo catturato, il flusso viene bloccato momentaneamente per eseguire prima il codice dell'advice.

Composizione di pointcut

Per creare regole di intercettazione più complesse, i pointcut possono essere combinati utilizzando i normali operatori booleani: AND, OR e NOT.

- OR (**||**): per intercettare metodi diversi (o anche tipi), ad esempio setX oppure setY:
  `call(void Point.setX(int)) || call(void Point.setY(int))`
- NOT (**!**): spesso usato insieme a within(), che identifica i join point dentro un modulo. Ad esempio:
  `call(void Point.setX(int)) && !within(C)`

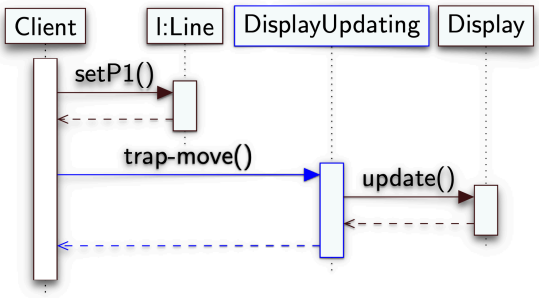
  cattura le chiamate al metodo solo se queste avvengono al di fuori della classe o dell'aspetto C

Esempio editor per figure

Applicando la programmazione ad aspetti (AOP) all'editor grafico, si crea un aspetto specifico chiamato **DisplayUpdating** per gestire l'aggiornamento dello schermo in modo isolato. All'interno dell'aspetto si definisce un unico **pointcut move()** che, usando una serie di operatori OR, raggruppa assieme tutte le operazioni che modificano le figure. A questo pointcut viene associato un **advice after()**. Di conseguenza, l'aggiornamento visivo scatta in automatico e le chiamate esplicite a Display.update() non sono più sparpagliate in giro per il metodi delle classi Line e Point.

```
public aspect DisplayUpdating {
        pointcut move():
                call(void Figure.moveBy(int, int))||
                call(void Line.setP1(Point)) ||
                call(void Line.setP2(Point)) ||
                call(void Point.setX(int)) ||
                call(void Point.setY(int));

        after() : move() {
                Display.update();
        }
}
```



Pointcut target

Oltre ad intercettare un evento, i pointcut possono fornire i valori del contesto in cui quell'evento si sta verificando. Il costrutto target() serve specificamente a fornire il riferimento all'oggetto reale su cui è stato invocato il metodo. Ad esempio, scrivendo:

`pointcut move(Figure fi) : target(fi) && call(void Figure.moveBy(int, int));`

il sistema cattura l'istanza specifica della figura che si sta muovendo e la salva nella variabile `fi`. Il valore di questo riferimento viene poi passato all'advice (ad esempio, `after(Figure f) : move(f)`), permettendo all'advice stesso di operare direttamente sull'oggetto.

Soluzione Aspect-Oriented

La soluzione finale per l'editor per figure unisce la collezione del contesto alla composizione logica. Viene definito un **pointcut move(Figure f)** che concatena con un AND l'estrazione del contesto **target(f)** all'insieme in OR di tutte le chiamate ai metodi di spostamento. L'**advice** associato **after(Figure fe)** prende in carico l'oggetto intercettato e invoca **Display.update(fe)**. Il risultato architetturale di questa soluzione è la totale modularità: l'intero concern di aggiornamento della grafica è descritto in un singolo modulo (l'aspetto **DisplayUpdating**). Se in futuro fosse necessario cambiare le logiche di aggiornamento visivo, tutti i cambiamenti si potranno fare esclusivamente in questo singolo aspetto, senza toccare il resto del sistema.

```
public aspect DisplayUpdating {
        pointcut move(Figure f):
                target(f) &&
                (call(void Figure.moveBy(int, int)) ||
                call(void Line.setP1(Point)) ||
                call(void Line.setP2(Point)) ||
                call(void Point.setX(int)) ||
                call(void Point.setY(int)));

        after(Figure fe) : move(fe) {
                Display.update(fe);
        }
}
```

Pointcut args

Il pointcut args() permette di conoscere e catturare i valori degli argomenti passati durante la chiamata del metodo intercettato. Ad esempio, definendo `pointcut move(int x, int y) : args(x, y) &&`

`call(void Figure.moveBy(int, int));`, i valori catturati x e y possono essere passati e utilizzati direttamente all'interno dell'advice associato (es. `after(int x, int y) : move(x,y)`).

Pointcut this

Il pointcut this(C) cattura tutti i join point in cui il riferimento this del linguaggio ad oggetti punta ad un'istanza della classe C o di una sua sottoclasse. A differenza del costrutto within(), this(C) non cattura l'esecuzione all'interno di classi annidate e non cattura i metodi statici. Se utilizzato con una variabile, come this(o), permette di fornire all'advice il riferimento dell'oggetto su cui avviene la cattura dell'esecuzione. Si comporta in modo analogo a target(), risultando praticamente identico ad esso se il pointcut è di tipo execution.

Advice

Gli advice implementano le azioni da eseguire in corrispondenza dei join point catturati. Esistono diversi tipi di advice in base al momento in cui intervengono:

→ **before()**: esegue il codice prima di procedere al join point.

→ **after()**: esegue il codice dopo il ritorno dal join point, indipendentemente da come questo avvenga.

→ **after() returning**: cattura il momento in cui l'esecuzione viene completata con successo e permette anche di estrarre il valore restituito tramite la sintassi **after() returning(Type o)**.

→ **after() throwing**: interviene quando viene lanciata un'eccezione, permettendo di catturarla specificandola con **after() throwing(Throwable e)**.

→ **around()**: esegue il codice al posto del join point, prendendo il controllo e decidendo se far continuare o meno l'esecuzione originale tramite l'invocazione di **proceed()**.

```
public aspect LoggerAsp {
        private Logger l = new Logger();

        pointcut logGet() : call(double Product.getPrice());

        after() returning(double r) : logGet() {
                l.writeLog("Price returned by getPrice");
                System.out.println("Price returned " + r);
        }

        after() throwing(Throwable e) : logGet() {
                System.out.println("Exception " + e);
        }
}
```

Advice around

L'advice around è particolarmente potente perché si sostituisce al metodo catturato. All'interno del suo corpo, si può leggere il valore dei parametri passati al metodo e modificare tale valore se non soddisfa certe condizioni, prima di passarlo a proceed(v) per far proseguire l'esecuzione.

```
public aspect Check {
        pointcut checkValore(int v) : // definizione pointcut
                args(v) &&
                (call(void Point.setX(int)) ||
                call(void Point.setY(int)));

        void around(int v) : // advice
                                        checkValore(v) { // pointcut colle
gato
                if (v < 9) v = 10; // corpo dell'advice
                proceed(v); // esegue metodo catturato
                // do something
        }
}
```

Inoltre, se le chiamate intercettate prevedono un valore di ritorno, l'advice around deve esplicitamente restituire questo valore utilizzando il costrutto `return proceed();`, oppure salvando il risultato di proceed() in una variabile per poi restituirla.

```
pointcut logGet() : call(double Product.getPrice());

double around() : logGet() {
        // do something
        return proceed();
}

double around() : logGet() {
        // do something
        double x = proceed();
        // do something
        return x;
}
```

Pointcut con wildcard

Le wildcard (caratteri jolly) si usano all'interno dei pointcut per catturare join point che presentano caratteristiche comuni senza doverli dichiarare singolarmente. I simboli principali sono:

- ***** : indica un numero qualsiasi di caratteri, escluso il punto.
- **..** : indica un numero qualsiasi di caratteri, incluso un qualunque numero di punti.
- **+** : indica qualunque sottoclasse o sottointerfaccia di un tipo specificato (ad esempio, Figure+). Ad esempio, la regola `public * Line.set*(..)` cattura qualsiasi metodo pubblico della classe Line il cui nome inizia con "set", mentre `void java.io.PrintStream.println(*)` cattura qualunque metodo println di PrintStream che ritorna void e prende un singolo argomento.

Esempio di uso di wildcard

Questi sono alcuni esempi pratici di applicazione delle wildcard per identificare insiemi di tipi:

- **∗Product**: indica qualsiasi tipo il cui nome termina con la stringa “Product”.
- **java.*.Date**: indica il tipo Date in qualsiasi package che si trova esattamente un livello sotto il package java.
- **java..***: indica qualsiasi tipo all’interno del package java o di un qualsiasi suo sotto-package.
- **javax..∗Model+**: indica tutti i tipi all’interno del package javax (e dei suoi subpackage) che terminano con “Model”, includendo anche tutti i loro sottotipi.
- **execution(* *.*(..))**: è una regola generica che cattura in assoluto tutte le esecuzioni di metodo all’interno del programma.